

Practice 6

Deadline: 2 weeks from now. Should be checked onsite (during labs).

Task 1

Walk through the "Part 3: Exception Flow" section in the tutorial. Run each case and explain the output to us.

Task 2

The `src.zip` in your `$JAVA_HOME` (`$JAVA_HOME` refers to JDK installation directory) contains the Java programming language source files for all classes that make up the Java Core API (that is, sources files for the `java.*`, `javax.*` and some `org.*` packages,). This source code is provided for informational purposes only, to help developers learn and use the Java programming language.

Let's practice `java.nio` and basic multithreading with the following tasks:

1. Traverse all `.java` files in `src.zip`.
2. Count the total number of lines in all `.java` files with:
 - a single-threaded implementation
 - a multi-threaded implementation.
3. Compare the performance of the two implementations.

1. Traverse all `.java` files recursively

We provided the `src.zip` in JDK 8 on BB that you can download and unzip locally. Write a method that scans the unzipped folder recursively and returns all `.java` files inside.

```
List<Path> collectJavaFiles(Path root)
```

You may use `Files.walk(root)` to traverse the directory tree recursively and returns a `Stream<Path>`.

2. Count total lines (single-threaded)

Write a method that takes a list of file paths and returns the total number of lines in these files.

```
int countTotalLines(List<Path> files)
```

You may check out APIs in `java.nio.file.Files` such as `Files.lines(Path)`.

3. Count total lines (multi-threaded)

Write a method that divides the given file list into `N` parts, creates `N` threads, and lets each thread compute its part's total line count. Finally, sum up all subtotals and return the final result.

```
int countTotalLinesMultiThreaded(List<Path> files, int nThreads){
    // Each thread's result stored here
    int[] partialResults = new int[nThreads];

    Thread[] threads = new Thread[nThreads];

    // TODO: start n threads

    // Wait for all threads to finish
    for (Thread t : threads) {
        t.join();
    }

    // TODO: sum up partial results
}
```

4. Compare performance

Write a `main` method to run the above methods. Compare the results and performance of single-threaded and multi-threaded implementations. You may use `System.currentTimeMillis()` to record start and end times.

```
long start = System.currentTimeMillis();
int total = countTotalLines(files);
long time = System.currentTimeMillis() - start;
```

Sample output: (Try use different number of threads and see how it affects the performance)

```
Total Java files in JDK 8 src.zip: 7701
(Single thread) Total lines: 2402508
(Single thread) Time: 12138 ms
(Multi threads) Total lines: 2402508
(Multi threads) Time: 3022 ms
```

Evaluation

The practice will be checked by teachers or SAs. What will be tested:

1. That you understand every line of your own code, not just copy from somewhere
2. That your program compiles correctly (javac)
3. Correctness of the program logic
4. That the result is obtained in a reasonable time

Late submissions after the deadline will incur a 20% penalty, meaning that you can only get 80% of this practice's score.